

**School of Computer Science and Artificial
Intelligence**

Lab Assignment # 15.5

Program : B. Tech (CSE)

Specialization :AIML

Course Title : AI Assisted Coding

Course Code : 23CS002PC304

Semester : VI

Academic Session : 2025-2026

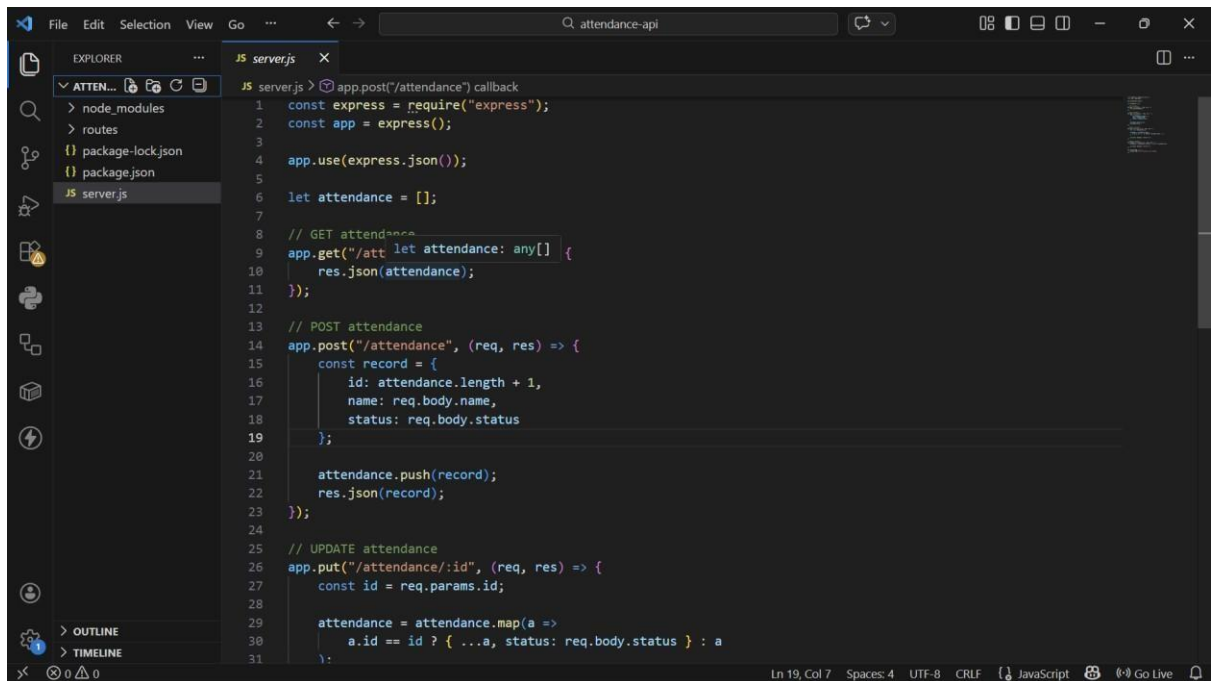
Name of Student : R. DEEKSHITH GOUD

Enrollment No. : 2303A52104

Batch No. : 33

Date :13/03/26

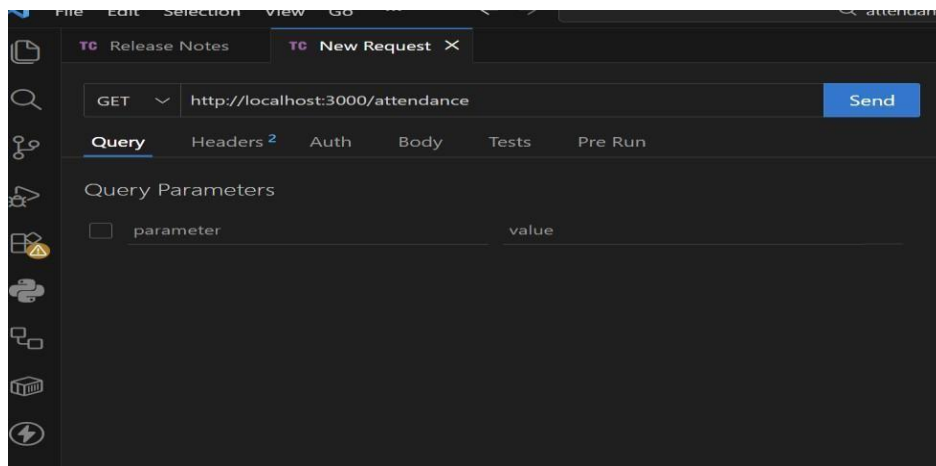
Task 1 – Student Attendance API



The screenshot shows a Visual Studio Code editor with a file named `server.js` open. The file contains JavaScript code for an Express.js application. The code defines a `GET /attendance` route that returns the current attendance array, and a `POST /attendance` route that adds a new attendance record to the array. The code is as follows:

```
1 const express = require("express");
2 const app = express();
3
4 app.use(express.json());
5
6 let attendance = [];
7
8 // GET attendance
9 app.get("/attendance", (req, res) => {
10   res.json(att let attendance: any[] {
11   });
12 });
13
14 // POST attendance
15 app.post("/attendance", (req, res) => {
16   const record = {
17     id: attendance.length + 1,
18     name: req.body.name,
19     status: req.body.status
20   };
21
22   attendance.push(record);
23   res.json(record);
24 });
25
26 // UPDATE attendance
27 app.put("/attendance/:id", (req, res) => {
28   const id = req.params.id;
29
30   attendance = attendance.map(a =>
31     a.id == id ? { ...a, status: req.body.status } : a
32   );
33 });
```

GET /attendance → View attendance records



o POST /attendance → Mark attendance

POST ⌵ http://localhost:3000/attendance Send

Query Headers ² Auth Body ¹ Tests Pre Run

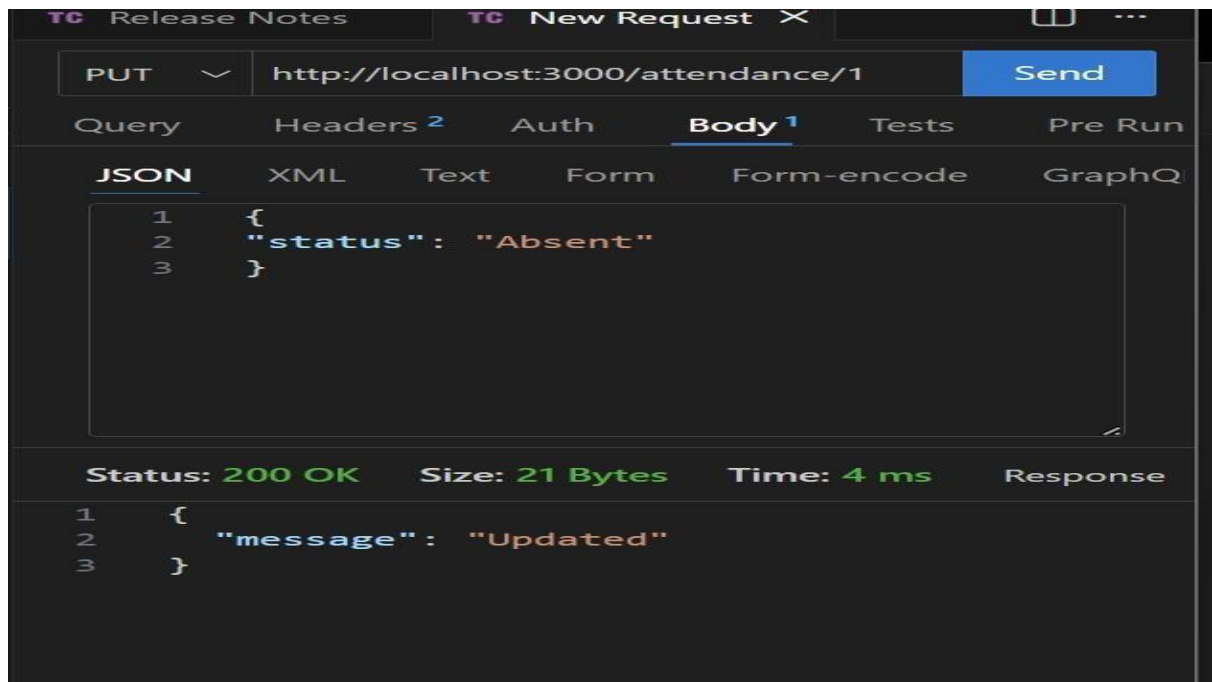
JSON XML Text Form Form-encode GraphQL

```
1 {
2   "name": "Anirudh",
3   "status": "Present"
4 }
```

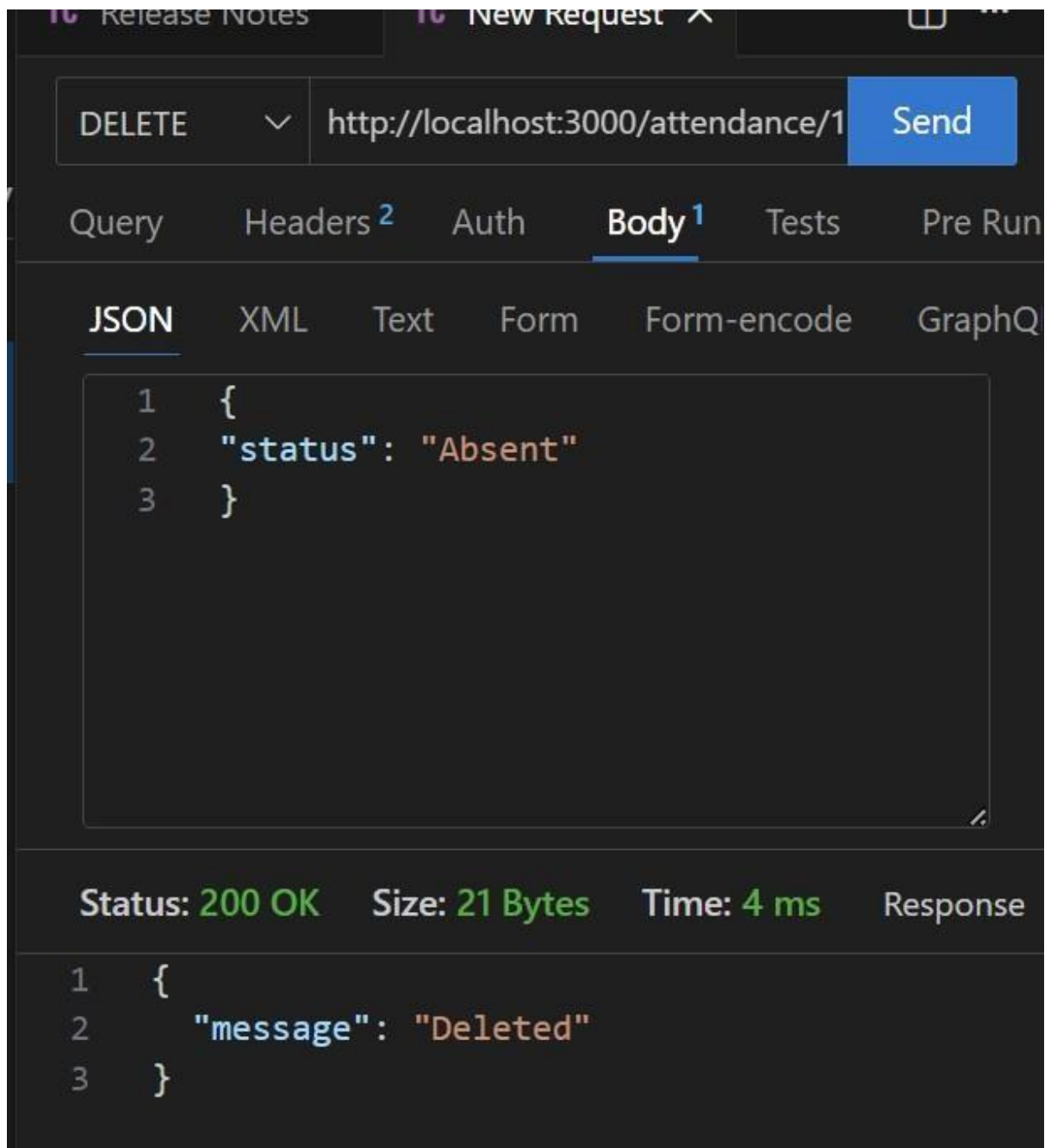
Status: 200 OK Size: 43 Bytes Time: 9 ms Response

```
1 {
2   "id": 1,
3   "name": "Anirudh",
4   "status": "Present"
5 }
```

o PUT /attendance/{id} → Update attendance record



o DELETE /attendance/{id} → Remove attendance entry



Task 2 – Inventory Management API

```
server.js
96
97 app.listen(3000, () => {
98   console.log("Server running on port 3000");
99 });
```

```
PS S:\3 2 sem\AIAssisted_2026\Assignment15.5\attendance-api> node server.js
Server running on port 3000
PS S:\3 2 sem\AIAssisted_2026\Assignment15.5\attendance-api> node server.js
Server running on port 3000
PS S:\3 2 sem\AIAssisted_2026\Assignment15.5\attendance-api> node server.js
S:\3 2 sem\AIAssisted_2026\Assignment15.5\attendance-api\server.js:6
app.get("/inventory", (req, res) => {
^
ReferenceError: app is not defined
    at Object.<anonymous> (S:\3 2 sem\AIAssisted_2026\Assignment15.5\attendance-api\server.js:6:1)
    at Module._compile (node:internal/modules/cjs/loader:1761:14)
    at Object..js (node:internal/modules/cjs/loader:1893:10)
    at Module.load (node:internal/modules/cjs/loader:1481:32)
    at Module._load (node:internal/modules/cjs/loader:1300:12)
    at TracingChannel.traceSync (node:diagnostics_channel:328:14)
    at wrapModuleLoad (node:internal/modules/cjs/loader:245:24)
    at Module.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:154:5)
    at node:internal/main/run_main_module:33:47

Node.js v24.13.0
PS S:\3 2 sem\AIAssisted_2026\Assignment15.5\attendance-api> node server.js
Server running on port 3000
```

GET /inventory → List all items

GET <http://localhost:3000/inventory> [Send](#)

Status: 200 OK Size: 2 Bytes Time: 9 ms

Query Headers 2 Auth Body 1 Tests Pre Run

Response Headers 6 Cookies Results Docs {}

1 [] [Copy](#)

Query Parameters

☐	parameter	value
--------------------------	-----------	-------

o POST /inventory → Add new item

POST `http://localhost:3000/inventory` **Send**

Status: **200 OK** Size: **35 Bytes** Time: **4 ms**

Query Headers² Auth **Body¹** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "name": "Laptop",
3   "stock": 10
4 }
```

Response Headers⁶ Cookies Results Docs {}

```
1 {
2   "id": 1,
3   "name": "Laptop",
4   "stock": 10
5 }
```

o **PUT /inventory/{id} → Update stock quantity**

JS server.js **PUT** `localhost:3000/inventory/1` **Send**

Status: **200 OK** Size: **27 Bytes** Time: **3 ms**

Query Headers² Auth **Body¹** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

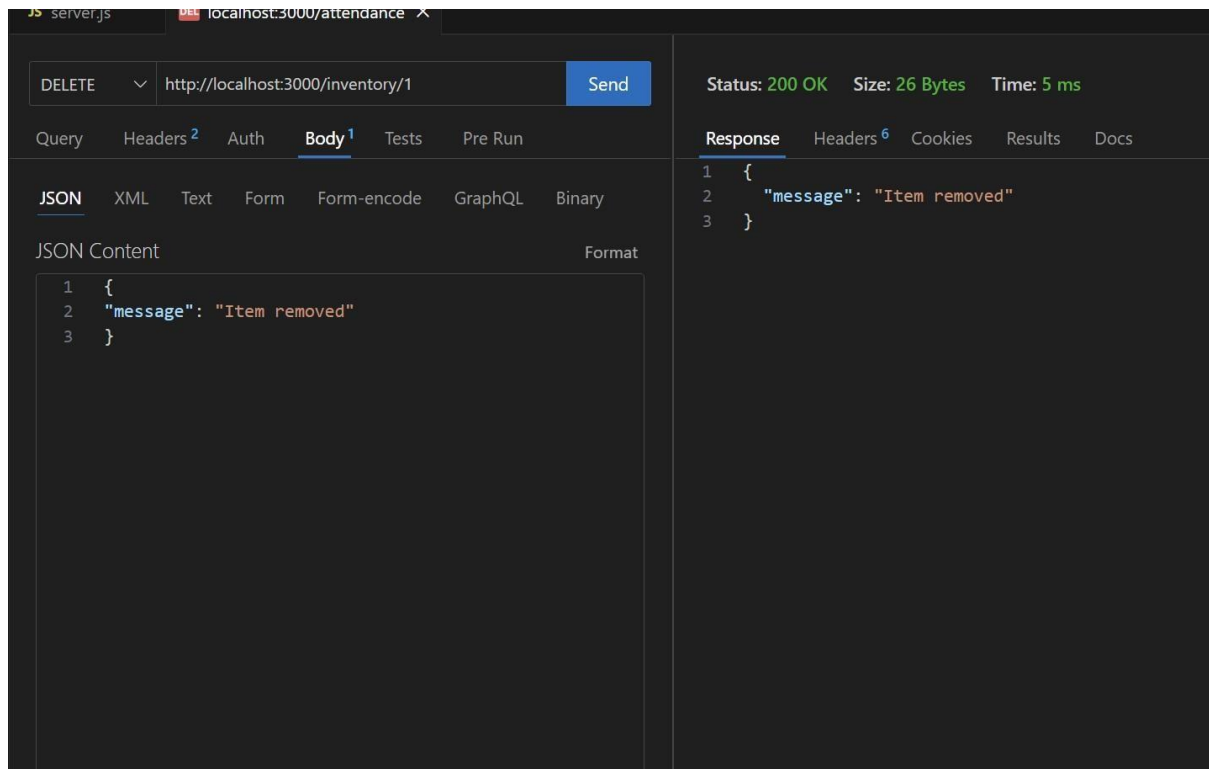
JSON Content Format

```
1 {
2   "stock": 20
3 }
```

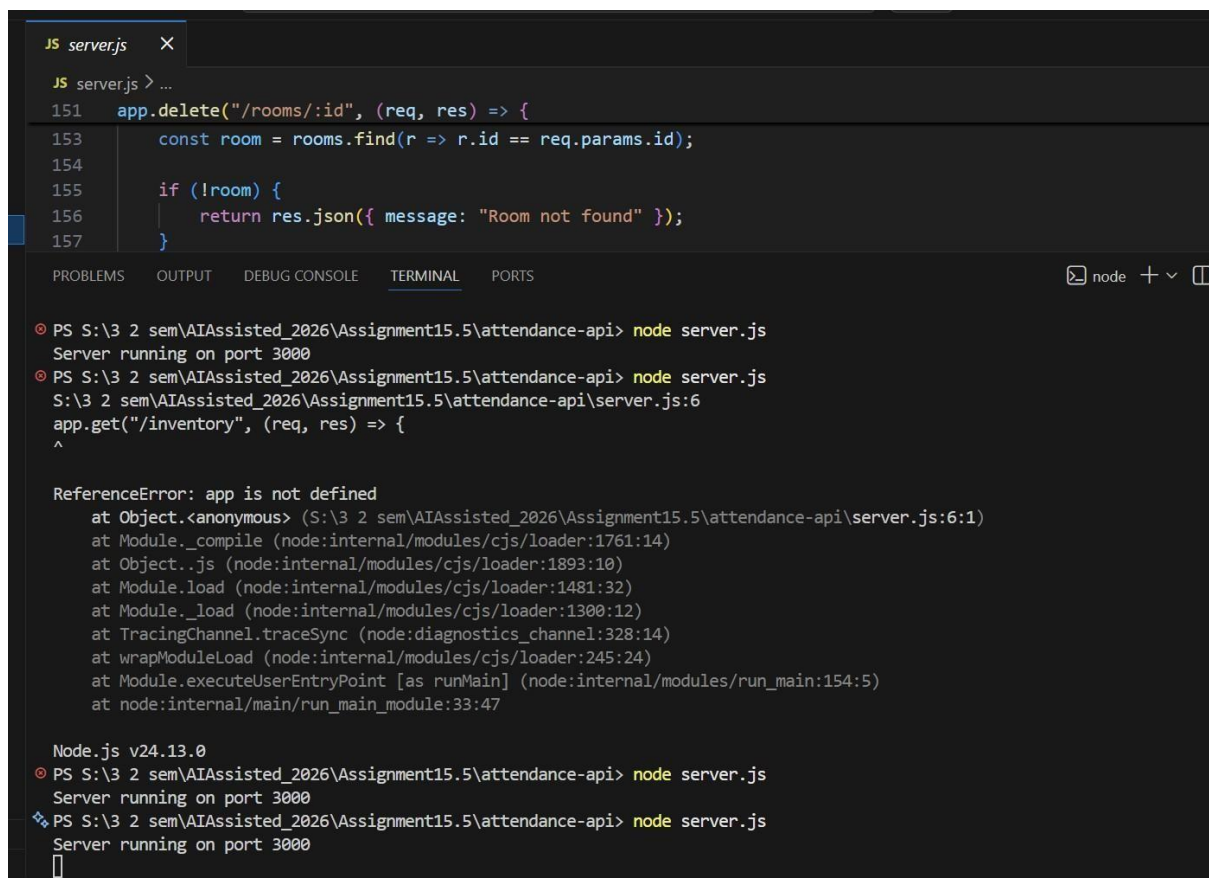
Response Headers⁶ Cookies Results Docs {}

```
1 {
2   "message": "Stock updated"
3 }
```

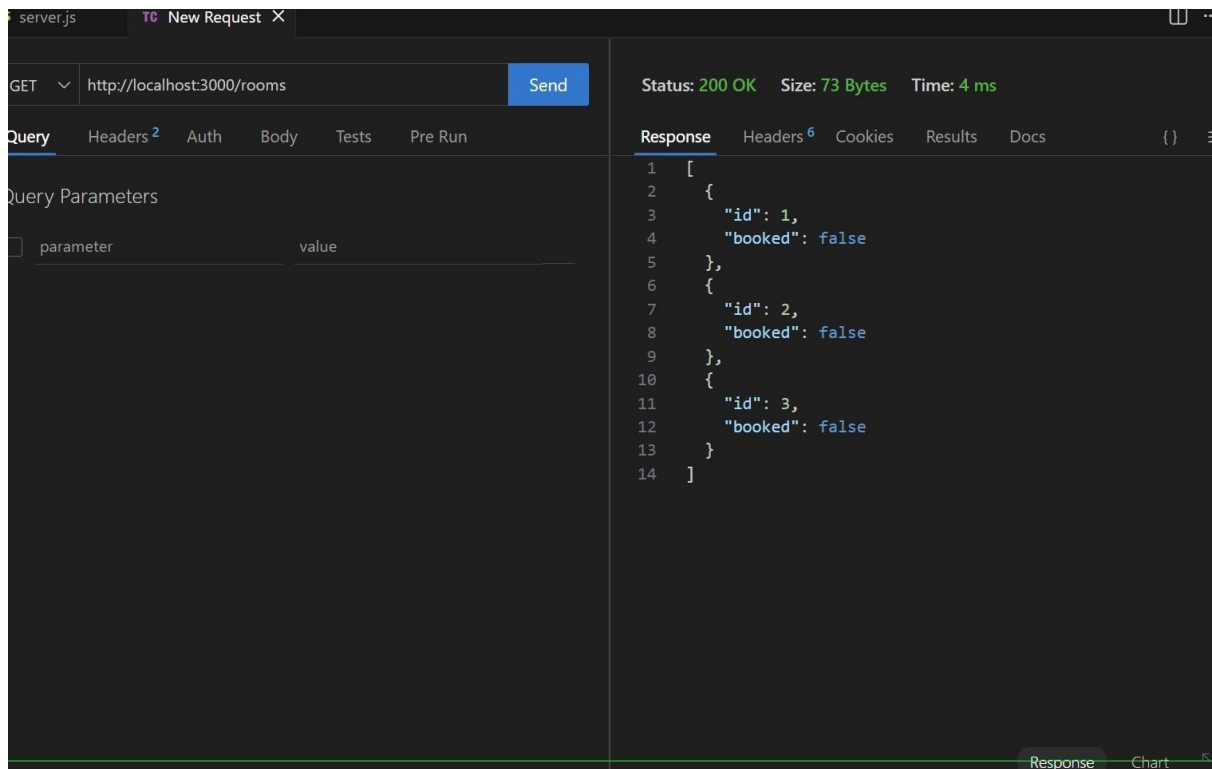
o **DELETE /inventory/{id} → Remove an item**



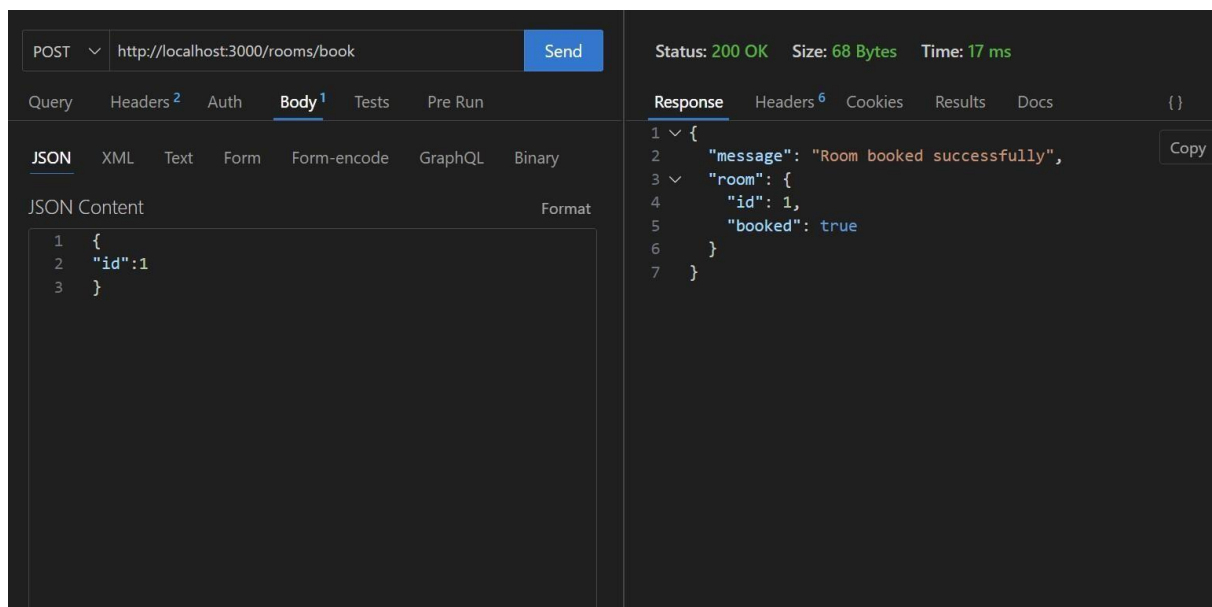
Task 3 – Hotel Room Booking API



o GET /rooms → View available rooms



o POST /rooms/book → Book a room



o GET /rooms/{id} → View booking details

GET ⌵ http://localhost:3000/rooms/1 Send

Status: 200 OK Size: 22 Bytes Time: 4 ms

Query Headers ² Auth Body ¹ Tests Pre Run

Response Headers ⁶ Cookies Results Docs

Query Parameters

☐	parameter	value
--------------------------	-----------	-------

```
1 {  
2   "id": 1,  
3   "booked": true  
4 }
```

o DELETE /rooms/{id} → Cancel booking

DELETE ⌵ http://localhost:3000/rooms/1 Send

Status: 200 OK Size: 31 Bytes Time: 2 ms

Query Headers ² Auth Body ¹ Tests Pre Run

Response Headers ⁶ Cookies Results Docs

Query Parameters

☐	parameter	value
--------------------------	-----------	-------

```
1 {  
2   "message": "Booking cancelled"  
3 }
```

Task 4 – Online Voting API

```
1 // Import express
2 const express = require("express");
3 const app = express();
4
5 app.use(express.json());
6
7 // Candidate data
8 let candidates = [
9   { id: 1, name: "Alice", votes: 0 },
10  { id: 2, name: "Bob", votes: 0 },
11  { id: 3, name: "Charlie", votes: 0 }
12 ];
13
14 // GET all candidates
15 app.get("/candidates", (req, res) => {
16   res.json(candidates);
17 });
18
19 // POST vote for candidate
20 app.post("/vote", (req, res) => {
21
22   const candidate = candidates.find(c => c.id == req.body.id);
23
24   if (!candidate) {
25     return res.json({ message: "Candidate not found" });
26   }
27
28   candidate.votes++;
29
30   res.json({
31     message: "Vote recorded"
```

o GET /candidates → List candidates

GET Send

Status: 200 OK Size: 103 Bytes Time: 8 ms

Query Headers² Auth Body Tests Pre Run

Response Headers⁶ Cookies Results Docs

Query Parameters

☐ parameter value

```
1 [
2   {
3     "id": 1,
4     "name": "Alice",
5     "votes": 0
6   },
7   {
8     "id": 2,
9     "name": "Bob",
10    "votes": 0
11  },
12  {
13    "id": 3,
14    "name": "Charlie",
15    "votes": 0
16  }
17 ]
```

o POST /vote → Cast a vote

The screenshot shows a REST client interface. The top bar indicates a POST request to `http://localhost:3000/vote` with a status of 200 OK, size of 73 Bytes, and time of 9 ms. The 'Body' tab is selected, showing the request body in JSON format: `{ "id": 1 }`. The 'Response' tab is also visible, showing the response body in JSON format: `{ "message": "Vote recorded", "candidate": { "id": 1, "name": "Alice", "votes": 1 } }`.

o GET /results → View voting results

The screenshot shows a REST client interface. The top bar indicates a GET request to `http://localhost:3000/results` with a status of 200 OK, size of 158 Bytes, and time of 3 ms. The 'Query' tab is selected, showing the query parameters. The 'Response' tab is also visible, showing the response body in JSON format: `{ "results": [ { "id": 1, "name": "Alice", "votes": 1 }, { "id": 2, "name": "Bob", "votes": 0 }, { "id": 3, "name": "Charlie", "votes": 0 } ], "winner": { "id": 1, "name": "Alice", "votes": 1 } }`.

Task 5 – Teacher Management API

```
JS server.js X T6 New Request
JS server.js > app.put("/teachers/:id") callback > message
37 app.put("/teachers/:id", (req, res) => {
45   teacher.name = req.body.name || teacher.name;
46   teacher.subject = req.body.subject || teacher.subject;
47
48   res.json({
49     message: "Teacher updated",
50     teacher: teacher
51   });
52 });
53
54 // DELETE /teachers/:id → Remove teacher
55 app.delete("/teachers/:id", (req, res) => {
56
57   const teacher = teachers.find(t => t.id == req.params.id);
58
59   if (!teacher) {
60     return res.status(404).json({ message: "Teacher not found" });
61   }
62
63   teachers = teachers.filter(t => t.id != req.params.id);
64
65   res.json({ message: "Teacher removed successfully" });
66 });
67
68 // Start server
69 app.listen(3000, () => {
70   console.log("Server running on port 3000");
71 });
```

o GET /teachers → List all teachers

The screenshot shows the VS Code interface with a REST client request and response. The request is a GET to `http://localhost:3000/teachers`. The response is a 200 OK status with a 2-byte body containing an empty array `[]`.

Method	URL	Status	Size	Time
GET	http://localhost:3000/teachers	200 OK	2 Bytes	11 ms

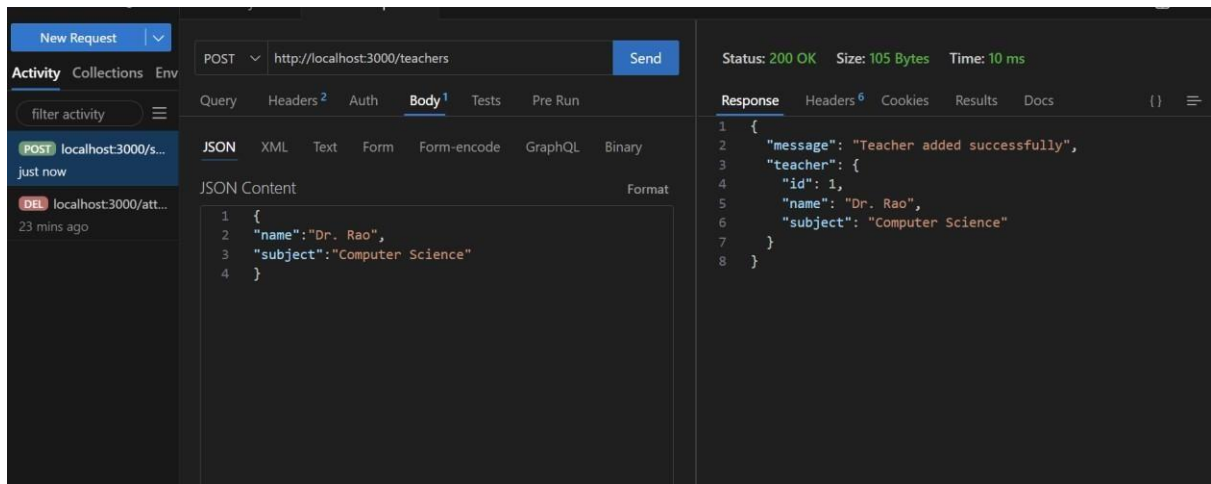
Query Parameters:

parameter	value
-----------	-------

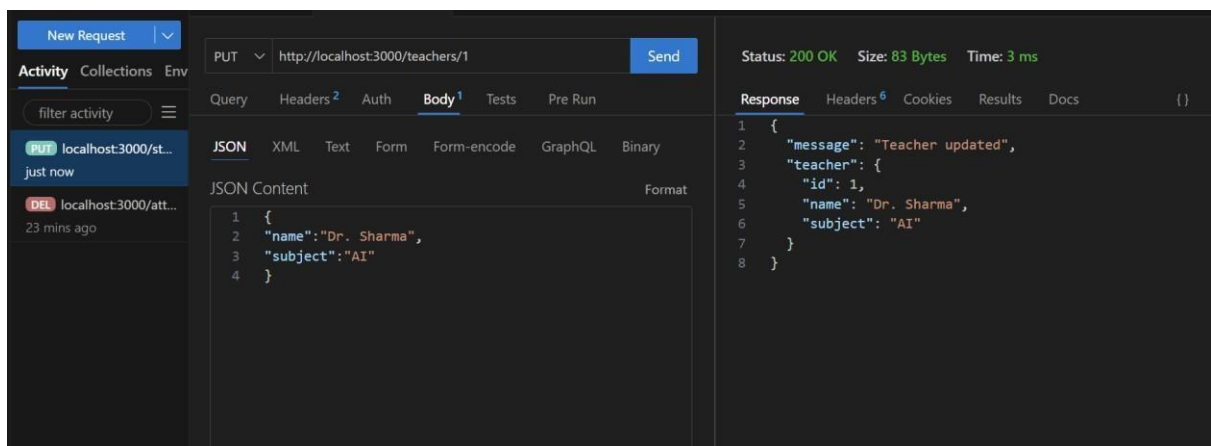
Response:

```
1 []
```

o POST /teachers → Add a new teacher



o PUT /teachers/{id} → Update teacher details



o DELETE /teachers/{id} → Remove a teacher

